

Sign Language Detection System

1) Project Overview

This project implements a Sign Language Detection System, applying Deep Learning and Computer Vision to recognize American Sign Language (ASL) alphabet gestures.

The system combines these techniques to:

1. Predict the ASL alphabet letter shown in an image using a CNN.
2. Recognize a set of known words by combining consecutively predicted letters.
3. Support both image upload and real-time webcam video input.
4. Restrict system operation to a specified time window.
5. Provide a graphical user interface (GUI) for image upload, webcam use, and result visualization.

2) Problem Statement

Standard sign-language recognition systems classify a single static hand-shape image into a letter. This project builds on that baseline by adding real-time video support and a known-word recognition layer on top of the letter-level predictions.

The objective of this project is to develop a machine learning system capable of:

1. Predicting the ASL alphabet letter shown in an image using a CNN.
2. Recognizing a fixed set of known words by combining consecutively predicted letters.
3. Supporting both image upload and real-time webcam video input.
4. Restricting system operation to a specified time window (6 PM-10 PM).
5. Displaying results through a user-friendly GUI.

3) Dataset

ASL Alphabet Dataset

Dataset Source:

Kaggle - [grassknoted/asl-alphabet](https://www.kaggle.com/grassknoted/asl-alphabet)

Labels are embedded directly in the folder structure, so no separate annotation step was required: each of the 29 class folders (A-Z, del, nothing, space) contains only images of that one sign.

Dataset Structure:

- 87,000 images used (the full dataset)
- 69,600 training images / 17,400 validation images (80/20 split)
- 29 classes: 26 letters plus del, nothing, and space

4) Methodology

1. Data Preprocessing

- Images loaded directly from labeled class folders.

- Images resized to 128 x 128 pixels.
- Pixel values normalized to [0,1] using a Rescaling layer inside the model.
- Dataset split into training and validation sets (80/20) with a fixed random seed.

2. Sign Classification Model

A Convolutional Neural Network (CNN) was trained to classify all 29 sign classes: a stack of Conv2D + BatchNormalization + MaxPooling2D blocks, then Dense layers.

Model Architecture:

- Random Rotation / Zoom / Translation (augmentation, training only)
- Rescaling Layer
- Convolution Layer (64 filters) + BatchNorm + Max Pooling
- Convolution Layer (128 filters) + BatchNorm + Max Pooling + Dropout (0.25)
- Convolution Layer (256 filters) + BatchNorm + Max Pooling + Dropout (0.25)
- Flatten
- Dense Layer (256 units)
- Dropout Layer (0.4)
- Dense Output Layer (29 units, Softmax)

Training configuration:

- Optimizer: Adam
- Loss: Categorical Cross-Entropy
- Epochs: 45 (EarlyStopping patience 8, not triggered - full budget used)
- Batch size: 64
- Learning rate scheduler: ReduceLROnPlateau (halved on plateau, reaching 3.125e-05 by the final epochs)

3. Known-Word Recognition

Instead of training a separate word-level model, known words are recognized with simple rule-based logic layered on top of the trained letter classifier:

1. Each predicted letter is appended to a running “word so far” string.
2. A predicted 'del' removes the last character; a predicted 'space' appends a space; a predicted 'nothing' is ignored.
3. If the resulting string matches an entry in a fixed known-word list (HELLO, YES, NO, THANKS, PLEASE, LOVE), it is flagged as a recognized word.

4. Real-Time Video Handling

For the webcam tab specifically:

1. Each frame is classified independently by the same CNN used for image upload.
2. Predictions below a confidence threshold (50%) are not added to the smoothing window.
3. The most common prediction across the last several confident frames is shown, to reduce frame-to-frame flicker from single noisy frames.

5. Time-Gated Operation

For each request to the system:

1. Check the current time against the configured operating window (6 PM-10 PM).
2. If outside the window, return a “closed” status instead of a prediction.
3. If inside the window, proceed with normal prediction.

5) GUI Development

Gradio was used to develop a graphical user interface.

Features:

- Upload a hand-sign image
- Preview the uploaded image
- Predict the ASL letter using the trained CNN
- Stream live webcam video and predict letters in real time
- Build up and display a spelled word from consecutive predictions
- Detect and flag recognized known words
- Log predictions and export the session log to Excel
- Restrict all predictions to the configured time window

6) Results

Sign Classification

Validation Accuracy: 99.94%

Validation Loss: 0.0021

Full classification report:

- Precision, recall, and f1-score of 1.00 for the majority of classes
- M, N, R, and U show a single point of deviation (0.99 instead of 1.00) in precision or recall
- Overall Accuracy: 1.00 (17,400 validation images)
- Macro avg / weighted avg: 1.00 / 1.00

Confusion Matrix

Strong diagonal with negligible off-diagonal values; no systematic confusion clusters between visually similar letters (e.g., M/N/S/T, U/V/R).

Real-Time Verification

Verified the Gradio app end-to-end: image upload, live webcam streaming, word-building (add/delete/space), known-word matching, time-gate enforcement outside 6 PM-10 PM, and Excel export of the prediction log.

7) Visualizations

Included in notebook/repository:

- Sample Letter-Sequence Grid (images for each letter of each known word)
- Training Accuracy Graph
- Training Loss Graph
- Confusion Matrix Heatmap

8) Technologies Used

- Python
- TensorFlow / Keras
- OpenCV
- NumPy
- Pandas
- Matplotlib
- Scikit-Learn
- Gradio
- openpyxl

9) Future Improvements

- Train on a larger, more diverse dataset (varied backgrounds/lighting)
- Use additional data augmentation to close the gap between validation and real-world accuracy
- Add recognition of complete words and sentences as single continuous gestures, rather than only spelled-out letters
- Support dynamic signs (e.g., J and Z, which are motion-based but currently classified from single static frames)
- Explore transfer learning for faster convergence and improved generalization
- Add confidence scores and prediction history to the GUI
- Deploy the application using Hugging Face Spaces, Streamlit, or Flask

10) Conclusion

This project successfully develops a CNN-based Sign Language Detection System capable of recognizing the American Sign Language alphabet from both static images and real-time webcam video. The developed system predicts ASL letters using a trained CNN, recognizes a fixed set of known words by combining consecutive predictions, restricts operation to a specified time window, and displays results through an interactive GUI. The model achieved 99.94% validation accuracy with near-perfect precision and recall across all 29 classes, and the known-word recognition, time-gating, and real-time smoothing behaviours were verified end-to-end through the Gradio interface.